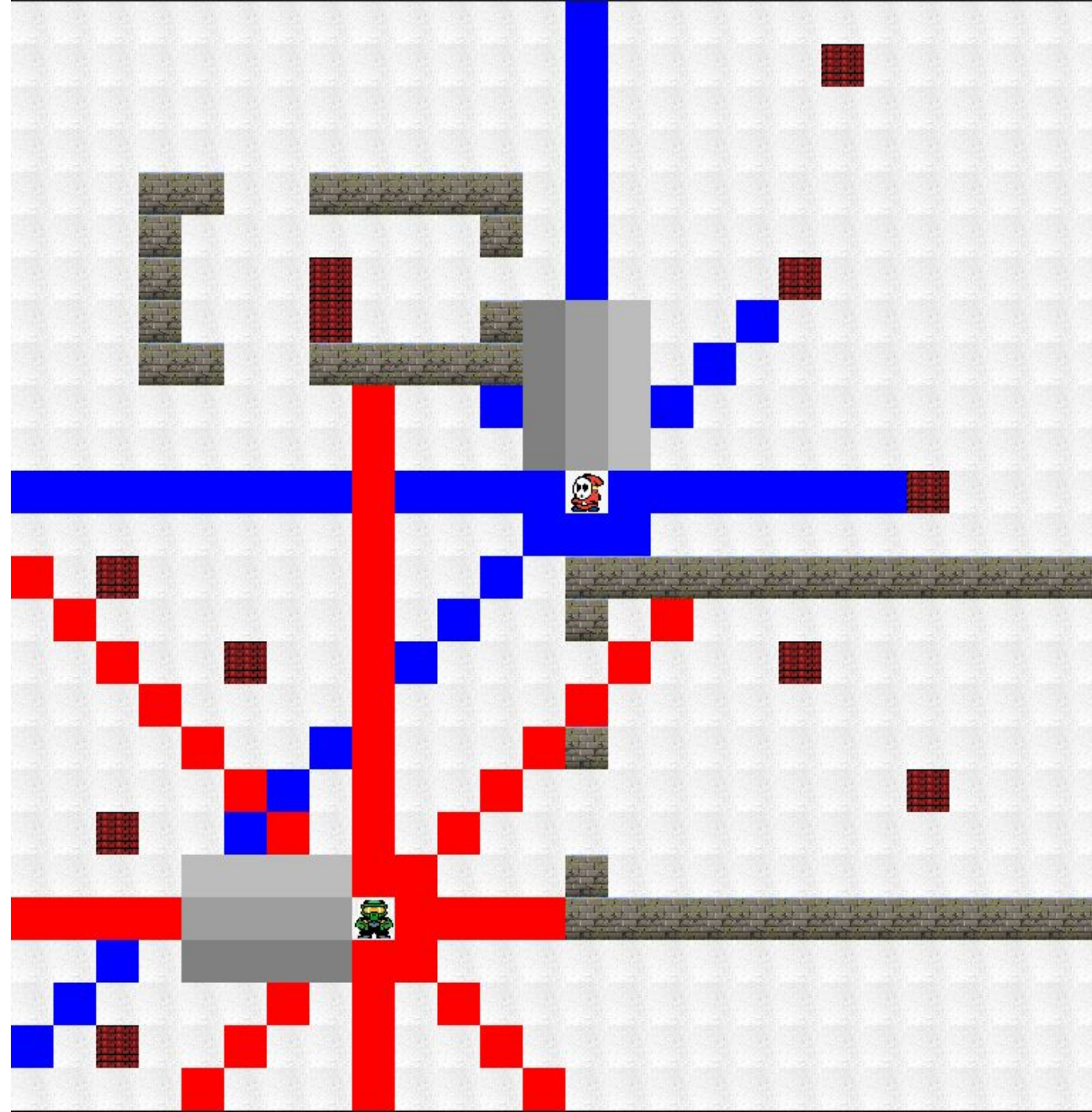


HIDE AND SEEK GAME

Candidates:

Kevin Gagliano
Pietro Pianigiani
Jacopo Andreucci



Project goal and reference

Development of the hide and seek game with pygame and agents empowered by Deep Q-learning.

Main goals:

- Create a graphically attractive game with multiple maps with increasing level of difficulty
- Implement agents based on reinforcement learning technique and the PyTorch framework
- Observing some intelligent behaviours from the players during the training phase

Main inspiration: multi-agent hide and seek game by OpenAI

<https://openai.com/research/emergent-tool-use>

Reinforcement learning

Reinforcement learning is a type of machine learning paradigm where an agent learns to make decisions by interacting with an environment. The agent learns to achieve a goal or maximize a cumulative reward by taking actions in the environment and observing the resulting outcomes.

Deep Q-learning

Deep Q-Learning is a type of reinforcement learning algorithm that combines the Q-learning algorithm with deep neural networks to learn optimal policies in complex environments with high-dimensional state spaces.

Main components:

- Q-learning
- Deep neural networks
- Experience replay
- Gradient based optimization

Game description

Key features of the game:

- Matrix structure of the game map (configurable from txt file)
- Map objects -> Floor (walkable tile), Movable wall (each player can grab it), Wall (immovable map component), Players (seeker and hider)
- Turn-based two player game with actions predicted by agents (neural models)
- Seeker wins when the hider is caught (based on the permanence of the hider in the seeker's view)
- Different reward criteria -> default, explore, explore2, distance, hybrid, smart evasion
- Possible player's actions -> move up, move down, move left, move right, grab/release movable wall, stand still

Player features

- Visual perception (view): list of object types with depth of 4 blocks and width of 3 with respect to the faced direction
- Short range perception (neighborhood): list of 3 blocks surrounding the player (left, back, right)
- Long range perception (lidar sensor): list of tuples (object type and distance) in the 8 directions spaced 45 degrees each



Code structure

- Objects.py: defines all the classes (and associated methods) present in the game map
- Game.py: defines core methods, rules and rewards for the game environment
- Models.py: defines neural network architectures and Q-trainers
- Agents.py: defines three main classes of agents and their respective methods
- Alpha_main.py: main file for training alpha agents
- Grid_search_alpha.py: grid search to discover the best hyperparameter configurations of alpha agents
- Beta_main.py: main file for training beta agents
- Play.py: file to load pre-trained agents and test them in game
- Util.py: defines functions for drawing learning curves and writing configuration files
- Plot_script.py: script for drawing the reward graph for each trained model

Neural networks

- QNet: plain multilayer perceptron with internal hidden layers architecture defined by the respective agent
- ConvQNet: convolutional neural network with internal architecture defined by the respective agent (hivemind)

Q-trainers

- Primary_Qtrainer: implement Q-Learning Algorithm using Bellman Equation and making no distinction between primary and target network
- Target_QTrainer: implement Q-Learning Algorithm using Bellman Equation and making distinction between primary and target network (target network is updated after a predefined number of games)

Alpha agents

- Our first generation of agents
- State representation : combination of positions, player's view, neighbourhood, distance
- Training methods: short memory (online), long memory (batch), replay memory (selective experience replay)
- Cons: needs to save memory file, limited exploration phase, computationally expensive

Hivemind agent

- Attempt to provide the agents all the possible information about the game state
- State representation: one-channel encoded frame of the entire grid
- Training methods: short memory (online), long memory (batch), replay memory (selective experience replay)
- Cons: needs to save memory file, limited exploration phase, computationally expensive

Beta agents

- State representation : combination of positions, player's view, neighbourhood, distance, lidar vision
- Training methods: long memory (batch) – short memory (implemented but not used)
- Pros: extended exploration phase, computationally less expensive, kept minimum degree of exploration

Small brain agent

- Beta agent with minimal state representation

Perfect seeker agent

- Just an algorithm capable of finding the hider in an empty map
- Employed to test hider agent

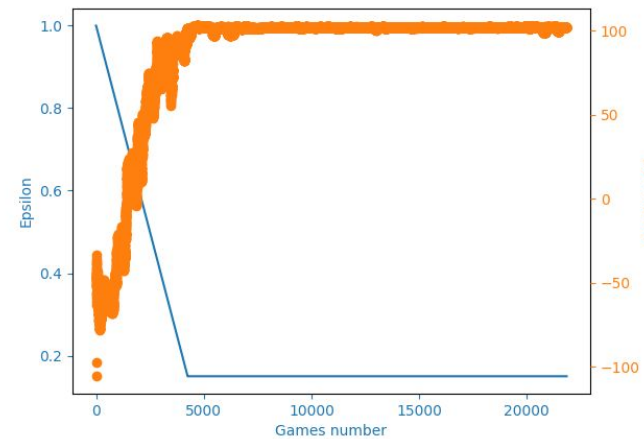
Achieved results

* Reward values depends strictly on the selected criterion

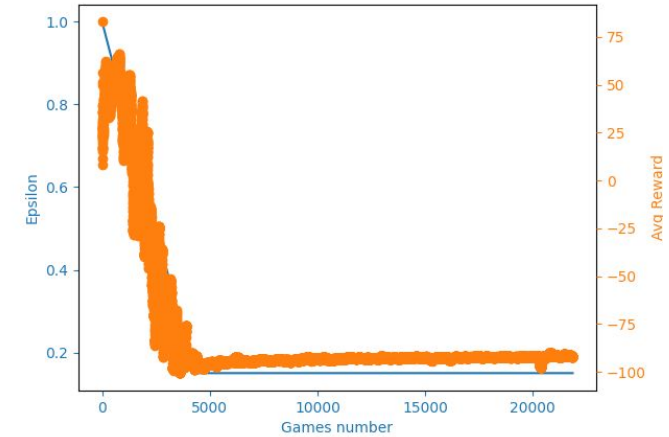
AGENT BETA 2

- Trained in full.txt map (the hardest one)
- Random spawn configuration (better generalization capability)
- Good tracking ability of the seeker
- Hider unable to develop counter strategies

SEEKER



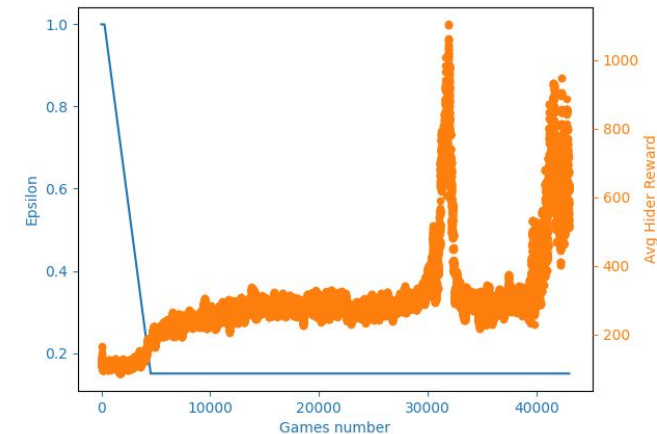
HIDER



AGENT SMALL BRAIN

- Trained in empty.txt map (the easiest one)
- Random spawn configuration (better generalization capability)
- Unstable behavior during learning
- Hider trained against the perfect seeker
- The only result where the hider somehow countered the seeker in the long run, but without any consistent outcome

HIDER



Considerations

- We succeeded in our goal of developing a good seeker, capable even in quite complex maps of finding the hider with a good success rate
- Hider did not demonstrate promising behavior during training and it will be necessary to rethink its internal structure
- Not every possible combination of hyperparameters and models were tested due to the amount of computation and time needed, the implemented methods might already be good enough to find a good hider agent and further experiments are required
- Convolutional neural network solutions (i.e. Hivemind) were not tested enough and may lead to better results if explored more deeply
- Short memory (online) and selective replay memory (selective batch sampling) training do not seem to improve the results given the same state representation and neural model

Future works

- Improvements of the state encoding and policies for rewards assignment
- Implementation of different reinforcement learning techniques (DQRN, PPO, ...)
- Tests on more complex structures of neural network architectures (LSTM, self-attention mechanism, ...)